CSE 211 — Theory of Computation

Variants of Turing Machines and TMs vs Computers

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Atif Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — simulation proofs and the true/false question
Lesson 16 — Multitape TMs and Their Simulation — definition, k-tape → 1-tape with 2k tracks, O(n²)
Lesson 17 — Nondeterministic and Restricted TMs — NTM → DTM (BFS), semi-infinite tape, two-way tape, left end, multistack, counters
Lesson 18 — Turing Machines and Computers — simulating a computer, polynomial time, the true/false answers
Quick Notes — TM Variants — revision bullets + key diagrams

NOTE Slide numbers refer to the PDF pages of Lecture - 6 (Turing Machine 3).pdf (68 pages). Machines in this note are described at the level of the slides (constructions); concrete one-tape machines are in the two previous TM notes. Practice provenance: Actual = taken from a TF paper; Adapted = TF question with changed details; Constructed = new question in TF style.

Exam Map — What the TF Papers Ask

Session	Questions on TM variants and simulations
2023-24	Q10 Multi-tape TM; how a one-tape TM with tracks and storage simulates it (15)
2022-23	Q11 T/F: DTM simulates n computer moves in p(n); a computer simulates n NTM moves in p(n) (15)
2021-22	Q10 T/F: multi-track 1-tape TM simulates multitape; a computer simulates n NTM moves in p(n) (15)
2020 (Jan term)	Q10 T/F: multi-track + storage simulates multitape; DTM simulates n computer steps in p(n) (15)
2019-20	Q10(b) How a one-tape TM with tracks and storage in the state simulates a multitape TM (7)
2017-18	Q10 T/F: multi-track 1-tape TM simulates multitape; DTM simulates NTM in poly moves; DTM simulates computer in poly moves (15)
2015-16	Q8(b) How multiple tracks and multiple tapes extend the TM; difference (10⅓)
2014-15	Q7(c) A nondeterministic TM is equivalent to a deterministic TM (9⅓)
2013-14	Q2(b) How can a TM recognize the left end of its tape? (7) · Q3(c) Every NTM has an equivalent deterministic TM (9⅓)
2012-13	Q3(a) Every multitape TM has an equivalent single-tape TM (8) · Q4(b) Every NTM has an equivalent deterministic TM (7)
2011-12	Q2(a) 2-tape TM deciding {wcw^R | w ∈ {a,b}*} using both tapes (8⅓) · Q2(b) A language is recognized by a two-way-infinite-tape TM iff by a one-way-infinite-tape TM (8)
2010-11	Q11(a) Single-tape TMs simulating: multitape TM, single-tape NTM, TM with 2-D tape (15)
2009-10	Q12(c) Three-tape TM subtracting two binary integers separated by ';' (first ≥ second) (11⅓)
2007-08	Q9 Short notes: multitape TM, multistack machine (10) · Q10 How a Turing machine can simulate a computer (10)
2006-07	Q13 Multitape and nondeterministic TMs: do they accept languages a basic TM cannot? (10)

EXAM FOCUS ★★★: a 15-mark 'briefly explain whether true or false' question in every paper since 2017-18 (2022-23, 2021-22, Jan-2020, 2017-18 — items: multi-track simulates multitape; DTM simulates NTM / computer in polynomial moves) and 7–15 marks on 'how a one-tape TM with tracks and storage in the state simulates a multitape TM' (2023-24, 2019-20). Before 2017: proofs that multitape and nondeterministic TMs add no power (every paper 2006-2015). Learn Lesson 16 and the table in Lesson 18.

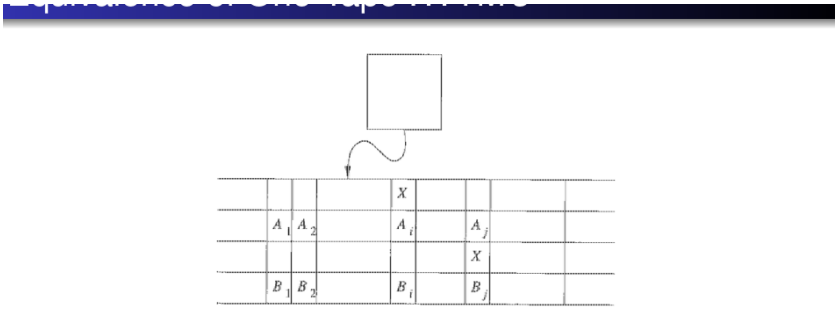
Lesson 16 — Multitape TMs and Their Simulation

Slides 1–22

1. The Multitape TM (sl. 4–10)

A finite control and k tapes, each with a blank and the same finite tape alphabet. Initially: the input on tape 1, all other cells blank, control in q0, head 1 at the left end of the input, the other heads anywhere. A move depends on the state and the k scanned symbols; in one move the control enters a new state, each tape gets a new symbol written, and each head moves L, R or S (stay) independently. Acceptance is by entering an accepting state.

2. Equivalence: One-Tape Machine with 2k Tracks (sl. 11–21)



- Suppose language *L* is accepted by a *k*-tape TM *M*.
- We simulate *M* with a one-tape TM *N* whose tape we think of as having 2*k* tracks.

Figure 16.1 — Two tapes simulated on one tape with four tracks (sl. 14)

EXAM-READY ANSWER Theorem: every language accepted by a multitape TM is RE (recognised by a one-tape TM). Proof (2012-13 Q3(a), 2019-20 Q10(b), 2023-24 Q10, 2015-16 Q8(b)). Let M be a k-tape TM; build a one-tape N whose tape has 2k tracks: track 2i holds the contents of tape i of M, track 2i − 1 holds a single head marker X at the cell where head i of M is. (Tracks = tape symbols that are tuples; storage in the state = N's state carries M's state, the count of head markers to the left, and the k scanned symbols.) To simulate one move of M: (1) N sweeps from the leftmost marker to the rightmost, and on meeting each marker stores the symbol scanned by that head in its finite control (it remembers how many markers lie to its left so it never gets lost); now N knows M's state and all k scanned symbols, hence M's move. (2) N sweeps back, at each marker writes the new symbol on that tape's track and moves the marker one cell L or R as M's move says (extending to the right when a head moves onto a not-yet-visited cell, writing blanks there). (3) N changes the recorded state of M. N's accepting states are those recording an accepting state of M. Thus N accepts exactly L(M).

3. Running Time (sl. 22)

After n moves of M the head markers cannot be more than 2n cells apart (each head moves at most n cells from its start). One simulated move costs at most 2n cells right to find all markers, at most 2n back while updating, plus ≤ 2k steps to reverse direction and extend the tape: ≤ 4n + 2k moves. Over n moves: $\Sigma(4i + 2k) = O(n^2)$ (k constant) — **one-tape simulation of a multitape TM costs a quadratic slowdown**.

4. Tracks versus Tapes (2015-16 Q8(b))

	Multiple tracks	Multiple tapes
What it is	one tape, each cell holds a tuple (one symbol per track)	k separate tapes, each with its own head
Heads	one head reads all tracks of a cell at once	k independent heads, may move in different directions, L/R/S
Alphabet	just a bigger tape alphabet — a notation	same alphabet per tape; the combination is in δ
Power	no extra power (relabel tuples as symbols)	no extra power in languages (Lesson 16 §2), but a quadratic speed-up
Typical use	check marks, small bookkeeping	copying, comparing, counters — simplifies a design

EXAM-READY ANSWER Multi-tape machine for {wcw^R | w ∈ {a,b}∗} (2011-12 Q2(a)): copy the part before c onto tape 2 (head 2 moves right with head 1), on reading c leave head 1 at the cell after c and move head 2 LEFT to the last copied symbol; then move head 1 right and head 2 left together, comparing symbols; accept if head 1 reaches the blank exactly when head 2 reaches the left blank with all comparisons equal; reject on any mismatch or on a second c.

EXAM-READY ANSWER 3-tape subtraction of binary integers 'x ; y' with x ≥ y (2009-10 Q12(c)): tape 1 input; copy x to tape 2 and y to tape 3, both right-aligned (head at the least significant bit) after moving to the right ends; subtract bit by bit from the LSB with a borrow in the state: x_i − y_i − b: write the result bit on tape 1 (or tape 2) and set the new borrow; when tape 3's number is exhausted treat y_i = 0; stop at the left ends; the difference remains on tape 2 and is copied back to tape 1. (A single-state-per-borrow table is as the addition machine with the table of x − y − b.)

Practice — Lesson 16

(a) Actual — 2012-13 Q3(a). (b) Actual — 2019-20 Q10(b), 2023-24 Q10. (c) Actual — 2006-07 Q13. (d) Actual — 2015-16 Q8(b). (e) Actual — 2011-12 Q2(a). (f) Actual — 2010-11 Q11(a) (multitape part).

(a) Show that every multitape TM has an equivalent single-tape TM. (8)

(b) What is a multi-tape TM? Explain how a one-tape TM with multiple tracks and storage in the state can simulate it. (7–15)

(c) What are multitape and non-deterministic TMs? Do they accept any language not accepted by basic TMs? (10)

(d) Describe how multiple tracks and multiple tapes extend the basic TM; discuss the difference. (10⅓)

(e) Design a two-tape TM that decides {wcw^R}. (8⅓)

(f) How does a single-tape TM simulate a multitape TM, and how long does it take? (part of 15)

✓ Solution

(a)–(b) EXAM-READY ANSWER in §2; the time analysis in §3 for the cost.

(c) Multitape: §1; NTM: Lesson 17 §1. Neither accepts any language outside the RE class: multitape → one tape by §2, NTM → deterministic by Lesson 17. They only make designs easier (and the simulations slower).

(d)–(f) Table in §4; machine for wcw^R in §4; time $O(n^2)$ in §3.

Lesson 17 — Nondeterministic and Restricted TMs

Slides 23–39

1. NTM (sl. 23–31)

δ(q, X) is a SET of triples {(q1, Y1, D1), …, (qk, Yk, Dk)}; the NTM picks any one triple entirely (it cannot mix a state from one, a symbol from another). It accepts w if SOME sequence of choices leads from the initial ID to an accepting state; other branches are irrelevant.

Theorem

If M_N is a nondeterministic Turing machine, then there is a deterministic Turing machine M_D such that $L(M_N) = L(M_D)$.

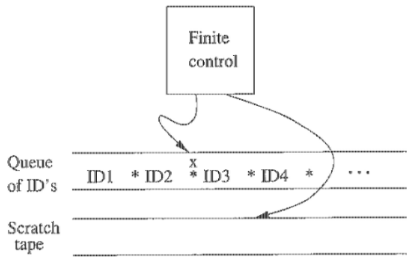


Figure 8.18: Simulation of an NTM by a DTM

PROOF: M_D will be designed as a multitape TM, sketched in figure.

Figure 17.1 — The deterministic simulator M_D: tape 1 holds a queue of IDs, tape 2 is scratch (sl. 26)

EXAM-READY ANSWER Theorem: every NTM M_N has a deterministic M_D with L(M_N) = L(M_D) (2012-13 Q4(b), 2013-14 Q3(c), 2014-15 Q7(c)). M_D is a multitape TM (then convert to one tape by Lesson 16). Tape 1 holds a sequence of IDs of M_N separated by ∗; one ID is marked 'current'; IDs to its left are finished. To process the current ID: (1) look at its state and scanned symbol; if the state is accepting, M_D accepts and stops. (2) Otherwise, if M_N has k choices for this (state, symbol), copy the ID with the help of tape 2 and append k copies at the END of the sequence on tape 1. (3) Modify each copy according to its own choice. (4) Return to the marked ID, erase the mark and mark the next ID to the right. Because new IDs are appended at the end (a queue — breadth-first search), every ID reachable in t moves is examined after a finite time; if M_N has an accepting branch of length t, M_D reaches it. If no branch accepts, M_D may run forever, which is allowed (both machines then reject/loop). Hence L(M_D) = L(M_N). (A stack — depth-first — would be wrong: an infinite branch would hide an accepting one.)

Cost: if every configuration has at most m successors, after n moves there are at most 1 + m + m² + … + m^n ≤ n·m^n IDs, so M_D may need **exponentially** more moves than M_N (sl. 31).

2. Restricted Machines (sl. 32–39)

Semi-infinite tape (sl. 33–36)

A TM whose tape is infinite only to the right simulates one infinite in both directions: use two tracks; the upper track holds cells X0, X1, … at and right of the start; the lower track holds X−1, X−2, … in reverse order. A ∗ on the leftmost cell's lower track is an end-marker preventing the head from falling off. The head moves right on the upper track = left on the lower track, and a 'bounce' at the marker switches track.

EXAM-READY ANSWER A language is recognised by a two-way-infinite-tape TM iff by a one-way-infinite-tape TM (2011-12 Q2(b)). (⇐) a one-way machine is a special case of a two-way machine (put a left-end marker or never move left of the start). (⇒) the two-track simulation above: the state is [q, U/L] where U/L says which track the head is on (storage in the state). Moves are reflected at the left end: the cell 0 has the marker ∗ on its lower track; reading it, the simulator swaps the direction and the track. Each step of the two-way machine takes one step, so the one-way machine accepts the same language in the same time.

EXAM-READY ANSWER How can a TM recognise the left end of its tape? (2013-14 Q2(b)) In the basic model the tape is infinite to the left, so the left end is not marked. Two standard tricks: (1) first shift the input one cell to the right and write a special marker (e.g. ▷ or #) in front of it, or write the marker on a second track of the first cell; the machine never moves left of the marker (when it reads ▷ it moves right). (2) For input that contains no blanks: the first blank to the left of the input is the left end — move left until a blank is read, then step right. Method (2) is what the machines of Lesson 12 use ('return to the blank').

Multistack and counter machines (sl. 37–39)

A k-stack machine is a deterministic PDA with k stacks (input comes from an input source, not the tape; one finite stack alphabet). **Two stacks simulate a TM**: the left stack holds the tape left of the head, the right stack holds the scanned symbol and everything to its right; a head move pops one stack and pushes onto the other. A **counter machine** replaces each stack by a counter (non-negative integer, only zero/non-zero tested; ±1). Every RE language is accepted by a two-counter machine. So: one stack < TM = two stacks = two counters.

Practice — Lesson 17

(a) Actual — 2012-13 Q4(b), 2013-14 Q3(c), 2014-15 Q7(c). (b) Actual — 2011-12 Q2(b). (c) Actual — 2013-14 Q2(b). (d) Actual — 2007-08 Q9. (e) Actual — 2010-11 Q11(a).

(a) Prove that every nondeterministic TM has an equivalent deterministic TM. (7–9⅓)

(b) Prove that a language is recognised by a TM with a two-way infinite tape iff it is recognised by a TM with a one-way infinite tape. (8)

(c) How can we recognise the left end of the tape of a TM? Explain. (7)

(d) Short notes: multitape TM; multistack machine. (10)

(e) Explain how single-tape TMs can simulate (i) a multitape TM, (ii) a single-tape NTM, (iii) a TM with a two-dimensional tape. (15)

✓ Solution

(a)–(c) EXAM-READY ANSWERS above.

(d) Multitape: Lesson 16 §1–§3. Multistack: §2 — a k-stack machine is a DPDA with k stacks; with two stacks it simulates a TM (left of head / right of head), hence accepts every RE language; with one stack it is just a (deterministic) PDA.

(e) (i) Lesson 16 §2. (ii) NTM → DTM above, then multitape → one tape. (iii) Two-dimensional tape: store the used rows linearly on one tape, rows separated by a marker, with a track marking the head position; moving left/right changes the offset inside a row; moving up/down jumps to the same offset in the previous/next row (counting symbols from the row marker by the 'mark and compare' technique); adding a new row/column shifts the data. Each 2-D step costs O(area) one-tape steps, polynomially many overall.

Lesson 18 — Turing Machines and Computers

Slides 40–68

A computer can simulate a TM; a TM can simulate a computer, in time at most polynomial in the number of computer steps (sl. 40). The notion of 'a computer' is informal, so the argument is informal.

1. Simulating a Computer by a TM (sl. 41–59)

Assume memory = an unbounded sequence of addressed words (addresses 0, 1, 2, …, binary), the program is stored in memory as words (first bits = opcode: copy, add, branch, …; the rest = addresses), each instruction touches a bounded number of words and changes at most one. The simulating multitape TM has: **tape 1** — the whole memory as $ addr1 ∗ contents1 # addr2 ∗ contents2 # … (markers $, ∗, #), **tape 2** — the instruction counter (a binary address), **tape 3** — a memory address / its contents, plus a scratch tape (sl. 49–51).

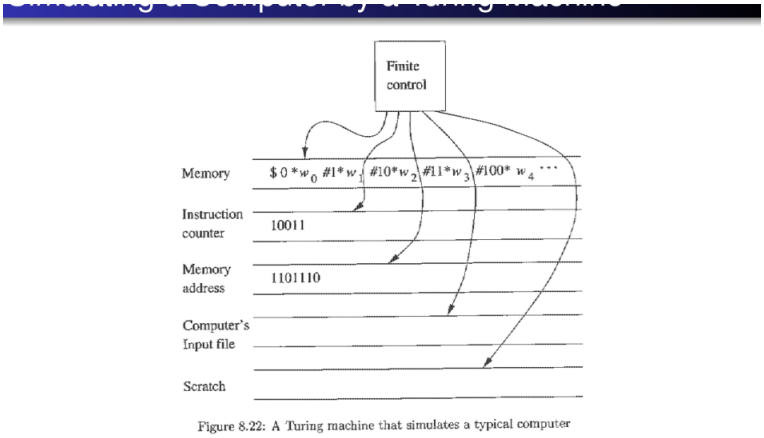


Figure 8.22: A Turing machine that simulates a typical computer

- This TM uses several tapes. It could be converted to a one-tape TM.

Figure 18.1 — The multitape TM that simulates a computer (sl. 48)

Instruction cycle (sl. 52–55): (1) search tape 1 from $ for the address equal to the counter on tape 2; (2) read that word's opcode and addresses; (3) if the instruction needs a memory value, copy the address to tape 3, mark the instruction's position on a second track of tape 1, find the address on tape 1, copy its value to tape 3; (4) execute; (5) add 1 to the instruction counter (or copy tape 3 into tape 2 for a jump) and repeat. Copy: find the second address and copy the value, **shifting over** (copy the rest of the tape to a scratch tape, write the new value, recopy) if its length changes — and if the address does not exist yet, create it at the end (sl. 56–57). Addition: ripple-carry from the right ends of the two values (sl. 58); jump: copy tape 3 to tape 2 (sl. 59).

2. Running Time (sl. 60–68)

- Multiplication would square the word length each step (2^(2^n) needs 2^n + 1 bits) — exponential. So assume each instruction yields a word at most one bit longer than its longer argument (addition is allowed, multiplication is not — m-bit multiplication = m additions and shifts, quadratic time).
- Theorem (sl. 64): if a computer has only instructions that lengthen words by ≤ 1 bit and a multitape TM can perform each on k-bit words in $O(k^2)$ steps, then the TM simulates n steps of the computer in $O(n^3)$ steps.
- Proof (sl. 65–68): after n steps no word is longer than c + n bits and the memory has ≤ d + n words; each word-address pair needs ≤ 2(c + n + 1) bits, so the tape has $O((c + n + 1)(d + n)) = O(n^2)$ cells; one instruction needs $O(n^2)$ steps (lookups, the $O(k^2)$ operation, shifting over). So n steps ⇒ $O(n^3)$.
- Converting the multitape TM to one tape squares the time again: **$O(n^6)$** (sl. 68).

EXAM-READY ANSWER Describe how a TM can simulate a computer (2007-08 Q10): memory, instruction counter and a scratch/address tape as above; the instruction cycle; 'shifting over' for variable-length words; polynomial overhead $O(n^3)$ on a multitape TM ($O(n^6)$ on one tape) if instructions grow words by at most one bit.

3. The True/False Question (★★★ every paper since 2017-18)

Statement	Answer	Reason
A one-tape TM with multiple tracks (and storage in the state) can simulate a multitape TM	TRUE	2k tracks hold the tapes and the k head markers; control sweeps to find the scanned symbols (Lesson 16 §2); polynomial (quadratic) slowdown
A deterministic TM can simulate n moves of a nondeterministic TM in p(n) moves for some polynomial p	FALSE (as far as is known)	the standard simulation explores up to m^n configurations (breadth-first): exponential. A polynomial simulation would give P = NP; no one has one, and it is believed impossible
A deterministic TM can simulate n moves of a conventional computer in p(n) moves	TRUE	Lesson 18 §2: $O(n^3)$ on a multitape TM, $O(n^6)$ on one tape (words grow at most one bit per instruction)
A (conventional) computer can simulate n moves of an NTM in p(n) steps	FALSE (as far as is known)	a computer is as fast as a DTM up to a polynomial, so this is the same question as the second statement (P vs NP)

Write each answer as: True/False — one sentence of reason — the time bound or the exponential count. Remember 'm^n configurations after n moves' for the NTM items.

Practice — Lesson 18

(a) Actual — 2017-18 Q10. (b) Actual — Jan-2020 Q10. (c) Actual — 2021-22 Q10. (d) Actual — 2022-23 Q11. (e) Actual — 2007-08 Q10.

(a) Briefly explain whether the following are true or false: (i) a one-tape TM with multiple tracks can simulate a multi-tape TM; (ii) a DTM can simulate n moves of an NTM in P(n) moves; (iii) a DTM can simulate n moves of a conventional computer in P(n) moves. (15)

(b) (i) A one-tape TM with multiple tracks and storage in the state can simulate a multi-tape TM. (ii) A DTM can simulate n steps of a conventional computer in p(n) steps. (7 + 8)

(c) (i) multi-track simulates multitape; (ii) a computer can simulate n steps of an NTM in p(n) steps. (15)

(d) (i) a DTM can simulate n steps of a conventional computer in p(n) steps; (ii) a computer can simulate n steps of an NTM in p(n) steps. (15)

(e) Describe how a Turing machine can simulate a computer. (10)

✓ Solution

(a)–(d) Use the table of §3: (a) T, F, T; (b) T, T; (c) T, F; (d) T, F. Support each with the reason column.

(e) EXAM-READY ANSWER in §2 (details §1).

Quick Notes — TM Variants

Revision points for Lessons 16–18.

L16 · Multitape

- k tapes, k heads, moves L/R/S. Simulation: 2k tracks (k contents + k head markers), state stores M's state + scanned symbols; sweep, update, sweep.
- Time: n moves of M → O(n²) one-tape moves (heads ≤ 2n apart; ≤ 4n + 2k per move).

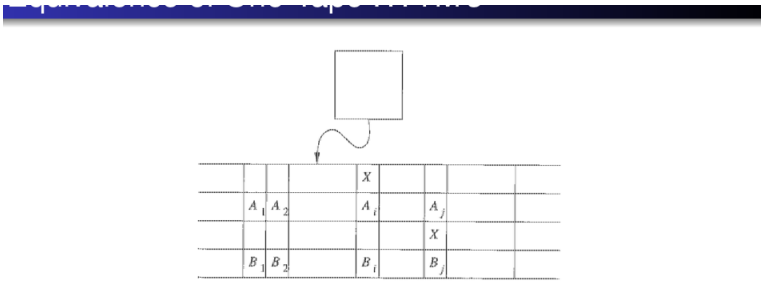
L17 · NTM and restricted

- NTM → DTM: multitape, queue of IDs (BFS), copy ID k times; ≤ n·m^n IDs → exponential.
- Semi-infinite tape: 2 tracks (right part / reversed left part), ∗ end-marker. Left end: marker or first blank.
- Two stacks = TM; two counters = TM; one stack < TM.

L18 · Computers

- Tapes: memory ($ addr ∗ val #), instruction counter, address/value, scratch. Cycle: find instruction, find operands, execute, increment.
- Words grow ≤ 1 bit/instruction: n steps → O(n³) multitape, O(n⁶) one tape.
- T/F: multi-track ⇒ multitape TRUE; DTM sims computer poly TRUE; DTM/computer sims NTM poly FALSE (as far as known).

Key Diagrams to Reproduce



- Suppose language L is accepted by a k-tape TM M.
- We simulate M with a one-tape TM N whose tape we think of as having $2k$ tracks.

k-tape → 2k tracks (sl. 14)